

This homework has 8 questions, for a total of 100 points and 8 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

For bonus/extra-credit questions, we will provide very limited guidance in office hours and on Piazza, and we do not guarantee anything about the difficulty of these questions.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up*.

(0 pts) 0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution: Shikha Nukala, desigirl

(10 pts) 1. **Self assessment.**

Carefully read and understand the posted solutions to the previous homework; you may also find the video “walkthroughs” on Canvas helpful. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn’t show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

Solution:

HW3 Question 4b

Solution:

function bottom-up(p)

//For the start of the algorithm, we need to take into account the base cases and output the results if p meets the requirements

if(p == 0) then return 1

if(p < 0) then return 0

//next for the bottom-up algorithm, initialize an array to hold all the possible ways to get to each score starting from the number one to the number p points. Make the first entry 1 as there is 1 way to get 0 points. Make the array size p + 1 so that the base case of 0 can be accounted for

int points[p + 1];

points[0] = 1;

//next have a for loop that goes from 1 to p points. For each number of points, we can calculate the ways to get those points by cumulatively adding the previous three ways to each.

for i = 1 to p

//add the previous way every time

```

points[i] = points[i] + points[i-1];
if(i ≥ 2)
points[i] = points[i] + points[i-2];
if(i ≥ 3)
points[i] = points[i] + points[i-3];
//now at index p the array points should contain all the ways to get to p points.
return points[p]

```

This algorithm is efficient because the time complexity is $O(p)$ which is linear. Since linear complexity algorithms are determined as efficient by the course, this algorithm is efficient.

This answer is partially incorrect because the true answer is that the algorithm described using the bottom-up format is not efficient for this class. Getting the solution in $\Theta(p)$ complexity is a linear complexity but in the context of the problem the input size of an integer has complexity $\Theta(\log p)$ thus the running time would turn out to be exponential and not linear as suggested by the algorithm complexity. Therefore it is not an efficient solution by the guidelines of the class. This error can be fixed by going over the fundamentals of runtime, input, and algorithmic complexity and reviewing how they change depending on the specifications of the algorithm.

2. Keeping things fresh with potpourri.

- (3 pts) (a) Consider the following function, which takes as input non-negative integers a and b that are powers of two.

```

1: function ALG( $a, b$ )
2:   if  $a = 1$  or  $b = 1$  or  $a = b$  then return 0
3:   if  $a > b$  then return ALG( $a/2, 2b$ )
4:   if  $b > a$  then return ALG( $2a, b/2$ )

```

Either find a valid potential function to prove that ALG halts on all valid inputs a, b , or show that no such function exists by giving an input on which ALG runs forever.

Solution: Looking at the function given we know that both a and b are both powers of two as given in the condition of the function. Therefore a good algorithm that combines the aspect of the power of two of both functions which can be done through logarithms. Therefore the potential algorithm can be $P(a, b) = \max(\log_2 a, \log_2 b)$. If we take this function then going from the first case we can see that the max of $\log_2 1, \log_2 1$ or if a and b are equal is going to amount to 0 as neither are greater than or less than each other. For the next case if a is greater than b we can return the max of $\log_2 a/2, \log_2 2b$ which becomes $\log_2 a - 1, \log_2 b + 1$ where we can see that since we know a is greater than b the potential decreases by 1. Then if we consider the third case as the max of $\log_2 2a, \log_2 b/2$ which becomes $\log_2 a + 1, \log_2 b - 1$ since b is greater than a the potential decreases by 1 here as well. Since in both cases the potential is

decreasing and the potential stops at the case when they are both equal then we can see that any input of a or b will have to halt the algorithm at some point and the algorithm cannot ever run into negative numbers for input.

- (3 pts) (b) Let $T(n)$ be the running time of the following function on an array of n entries. Write a recurrence for $T(n)$, and *briefly* justify your answer.

```

1: function FOO( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1] + 1$ 
3:    $x = 2 \cdot \text{FOO}(A[1, \dots, n/2])$ 
4:    $y = 3 \cdot \text{FOO}(A[n/2 + 1, \dots, n])$ 
5:    $tmp = 0$ 
6:   for  $i = 1, \dots, n$  do
7:     for  $j = 1, \dots, n$  do
8:        $tmp \leftarrow tmp + A[i] + A[j] - x + y$ 
9:   return  $tmp$ 

```

Solution: From the function we can see that there are two arrays which recursively are called in the function again. Both arrays are of $n/2$ size so the recurrence factor of these arrays is $T(n/2)$. For the first recursive call the call is multiplied by 2 so the recurrence factor of this call is also multiplied by 2 to get $2T(n/2)$. The second call is multiplied by 3 so we can write the recurrence factor as $3T(n/2)$. Lastly we can see that there are two for loops that are nested iterating from 1 to n , so the time complexity is exponential and can be written as $O(n^2)$. Therefore using the recursive calls and loops in the code, we can write the complete recurrence relation as $T(n) = 2T(n/2) + 3T(n/2) + O(n^2)$. Therefore in accounting for the recursive calls done on each half of the input array and the nested for loop over the entire array, we get this recurrence relation.

- (4 pts) (c) Suppose there are algorithms A and B, where $T_A(n)$ and $T_B(n)$ are respectively the worst-case running times of A and B on inputs of size n . Suppose that T_A satisfies $T_A(n) = 2T_A(n/2) + T_B(n)$, and $T_B(n) = O(n^2)$ but $T_B(n) \neq O(1)$. Select all bounds for $T_A(n)$ that *could possibly hold* (for some choice of $T_B(n)$ satisfying the above bounds):

- A. $\Theta(n^3)$
- B. $\Theta(n^2)$
- C. $\Theta(n \log n)$
- D. $\Theta(n)$
- E. $\Theta(1)$

Choose one answer you selected as possible, and give a specific choice of $T_B(n)$ that would yield that result. You do not need to provide any justifications, other than why your choice of $T_B(n)$ yields the claimed solution for $T_A(n)$.

Solution: We know that $T_B(n)$ cannot be bound by $\Theta(1)$ because it is not a constant term thus since it is a part of $T_A(n)$ it cannot hold true for $T_A(n)$ as well. For the rest

of the bounds all can potentially hold true for some $T_B(n)$ as $O(n^2)$ is the higher bound for $T_B(n)$ which means $T_A(n)$ could reflect any one of the bounds as long as $T_B(n)$ and $2T_A(n/2)$ account for them. Using the potential bounds if the bound of $T_A(n)$ is $\Theta(n^2)$ then the possible $T_B(n)$ could be $T_B(n) = n^2$ as the highest determining complexity factor of $T_A(n)$ must be an exponential growth of 2 as the recurrence in $T_A(n)$ only reflects a $\Theta(n)$ complexity.

3. Counting certain cupcakes.

Lily has started a new job at Bakehouse 46 in Downtown Ann Arbor! Her responsibilities in opening up the store everyday include placing the store's cupcakes in the display case. The cafe makes several different flavors of cupcakes, and various numbers of each flavor. The manager requires that for each flavor, either *all* or *none* of the cupcakes of that flavor are displayed. Lily wants to determine whether it is possible to *completely* fill the display case with cupcakes while following this rule.

We formalize the problem as follows. We are given an array of non-negative integers $S[1, \dots, n]$, where each $S[i]$ represents the number of cupcakes of the i th flavor, and a non-negative integer K representing the total number of cupcakes the display case can hold. We wish to determine whether there is a set of flavors that have exactly K cupcakes in total.

Given the array S , for suitable integers i, m (in ranges for you to determine) define $D(i, m)$ to be the Boolean value indicating whether there a subset of the first i flavors having exactly m cupcakes.

(10 pts)

- (a) Derive a recurrence relation, including base case(s), for $D(i, m)$ that is suitable for a dynamic-programming solution to this problem. Briefly justify its correctness.

Solution: To first determine the base cases with i being the number of flavors of cupcakes in the subset and m being the total number of cupcakes in the subset, we can see that if either i or m is 0, then we have either reached a maximum if the total required is 0 or there are no flavor combinations to achieve this goal. To write this we can say that the base cases are:

$D(i, 0)$ for all i in the case. This means that if the number of cupcakes required is 0 then none of any flavor will fulfill this.

$D(0, m)$ for all m in the case. This means that if there are no flavor combinations that make the required number m then 0 choices of i will work.

For the dynamic programming solution we consider two cases: if we take out the last flavor i from the subset then the expression $D(i, m)$ should hold true because exactly m flavors could be achieved before adding the last flavor. The other case is if we take out the last flavor i from the subset then the expression $D(i, m)$ should hold true only if m is equal to m minus the number of cupcakes in the last flavor that can complete the requirement. To write these conditions out we can say the recurrence relation is:

$D(i - 1, m)$ holds true when there are enough cupcakes without the last flavor

$D(i - 1, m - S[i])$ holds true when the last flavor is excluded the number of cupcakes in the last flavor should complete the requirement.

This holds true for all values of i since without any one of the flavors in the subset, it should either hold true that the flavor will assist the number m needed or will not be required in the number m needed.

- (5 pts) (b) Give pseudocode for a (bottom-up) dynamic programming algorithm that solves this problem, and analyze its running time.

Solution:

Pseudocode:

```
//take in the array and the total number of cupcakes the display can hold
countCupcakes(S, m)
//get the number of flavors i from the array
int i = getLengthOf(S)
//set an unordered map of values where the key represents the number of flavors in a
subset and the value represents the total number of cupcakes in the total subset
mapSubsets{int<i>:int<i>}
//use a for loop to go through the base cases and set the subsets of each singular flavor
if m were 0 for(1 to i flavors)
set all D(i, 0) values to true
//next we go through all flavors
for(1 to i flavors)
//go through all the cupcake numbers possible
for all 1 to m cupcakes
check if we are at m cupcakes and if so then compute  $D(i - 1, m - S[i])$  if not then
compute  $D(i - 1, m)$  and set these values to the mapSubsets at flavor i
see if there is a true value in mapSubsets that amounts to m cupcakes for i flavors and
return true if found.
```

The time complexity for this is $O(n * k)$ as the two for loops running in this algorithm go through two different sizes of values.

4. Dynamic programming for dynamic shortest paths.

Graphs that arise in real applications are often not fixed, but can change over time; such graphs are called “dynamic.” (This is a totally different use of the word “dynamic” than in “dynamic programming”. Computer scientists sometimes name things in confusing ways!) For example, roads and intersections can be added to or deleted from the road network, computers can be added to or removed from the Internet, and people can (un)follow each other on social networks. For many problems of interest, there are algorithms for dynamic graphs that are faster than just re-computing answers “from scratch” whenever the graph changes. In this problem you will give such algorithms for shortest-path problems.

Let $G = (V, E)$ be a weighted directed graph with $n = |V|$ vertices, where the weight of each edge (u, v) is denoted $\ell(u, v)$. (There is no negative-weight cycle in G .) Suppose that we have already computed the all-pairs distance table D_G of G . That is, for each vertex pair $u, v \in V$, $D_G(u, v)$ stores the distance (i.e., the length of a shortest path) from u to v in G .

Now, a new vertex v_{new} is added to the graph, together with its incident edges. Let $G_{new} = (V \cup \{v_{new}\}, E \cup E_{new})$ denote the updated graph, where E_{new} consists of all the incoming and outgoing edges for v_{new} . (There is no negative-weight cycle in G_{new} .) We aim to compute the updated distance table $D_{G_{new}}$ of G_{new} .

A straightforward approach is to compute $D_{G_{new}}$ from scratch using the Floyd-Warshall algorithm, in $\Theta(n^3)$ time. But we want to do better by exploiting the fact that we already have D_G . In this problem, you will obtain a faster $O(n^2)$ -time algorithm.

- (20 pts) (a) Write expressions for $D_{G_{new}}(v_{new}, u)$ and $D_{G_{new}}(u, v_{new})$ that hold for all $u \in V$, where the expressions are in terms of the already-known quantities $D_G(y, z)$ for $y, z \in V$, and $\ell(y, z)$ for $y, z \in V \cup \{v_{new}\}$. Evaluating your expressions should take $O(n)$ time for each vertex u , for a total of $O(n^2)$ time. Justify the correctness of your expressions and the evaluation time.

Hint: Consider why the Bellman-Ford algorithm is correct.

Solution: The Bellman-Ford algorithm is correct in the sense that if there is a shorter path from one vertex to another, then the algorithm will go down that path. Therefore if we already know the distance D_G from any to vertices then we just needed to find the minimum paths from

$$D_{G_{new}}(v_{new}, u) = \min(DG(v_{new}, u), \min_{y \in V} DG(y, u) + \ell(y, v_{new}))$$

$$D_{G_{new}}(u, v_{new}) = \min(DG(u, v_{new}), \min_{y \in V} DG(u, y) + \ell(v_{new}, y))$$

Each of these operations takes $O(n)$ time because for the V values the minimum is computed once and since there are n vertices in V the total time needed to compute would be $O(n^2)$.

- (10 pts) (b) Write an expression for $D_{G_{new}}(u, v)$ that holds for all $u, v \in V$, where the expression is in terms of the already-known quantities listed in the previous part, as well as the quantities you computed in the previous part. Evaluating your expression for all $u, v \in V$ should take a total of $O(n^2)$ time. Justify the correctness of your expression and the evaluation time.

Observe that by combining the two parts, we have computed the entire new distance table $D_{G_{new}}$ of G_{new} in $O(n^2)$ time.

Hint: Consider why the Floyd-Warshall algorithm is correct.

Solution: The Floyd-Warshall algorithm checks how to reach a certain vertex from one vertex through an intermediate vertex path. Therefore to solve this problem we know the distance from v to u for all quantities and can use this to compute the distance for intermediate use of v_{new} to the rest. Therefore we can write the solution as

$$D_{G_{new}}(u, v) = \min(DG(u, v), \min_{y \in V} DG(u, y) + D_{G_{new}}(y, v))$$

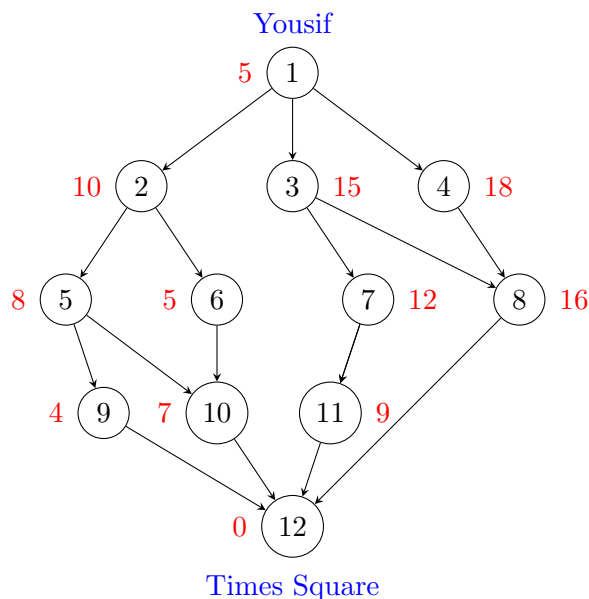
Since there are n number of choices for both v and u in this case the algorithm takes $O(n^2)$ time to compute this. Therefore the total complexity should be the same.

5. Maximizing money made in Manhattan.

To manage the surge in New Year’s Eve tourism, the New York City government has implemented traffic control measures: all roads have been made one-way; to prevent accumulating traffic, all intersections are reachable and there are no loops or dead ends (except at Times Square); and all tourists must use ride-shares. As the midnight countdown approaches, tourists at various intersections are “bidding” on rides to Times Square to witness the iconic Ball Drop, by posting the prices they are willing to pay for rides.

Yousif, a limousine driver, will drive through the city to Times Square, picking up all the tourists at each intersection along the way. (His limousine has unlimited passenger capacity and fuel.) Yousif’s goal is to find a route to Times Square that maximizes the total money he earns.

We model this problem mathematically as follows (see the graph below for an example). The city’s road network is given by a directed acyclic graph (DAG) $G = (V, E)$ with n vertices given in topological order as $V = \{1, \dots, n\}$ (representing intersections), and m directed edges $(i, j) \in E$ where $i < j$ (representing road segments). Every vertex $i < n$ has at least one outgoing edge (there is only one dead end), and every vertex $j > 1$ has at least one incoming edge (all intersections are reachable). For each vertex i there is an associated price $p_i \geq 0$ that the tourists at intersection i are willing to pay (in total). Yousif wishes to find a route from vertex 1 (his starting location) to vertex n (Times Square) that maximizes the total price along the route.



(2 pts)

- (a) Consider the following “greedy” algorithm for this problem: at each step, move to an adjacent vertex that has the maximum price, until arriving at Times Square. What total price will this algorithm yield for the above graph? (The price for each vertex is shown next to it, in red.) Is it optimal, and why or why not?

Solution: If we simply start from Yousif and take the greedy route then we start with 1:5 dollars the 4:18 dollars then 8: 16 dollars then 12: 0 dollars so the total profit

Yousif gets is $5+18+16+0 = 39$ dollars. This is not the most optimal yield from the graph because there is another route that can yield a higher return.

- (4 pts) (b) Briefly but clearly describe a brute-force algorithm for this problem, which may run in exponential time in the number of vertices n . You do not need to give pseudocode, justify correctness, or analyze the running time.

Solution: A brute force algorithm would be to calculate or store all possible routes that can occur from point 1 to point n by going down every single route and calculating the cost. Therefore this would need nested iteration to consider the different paths and conditions needed to be checked at each path which would lead to exponential time.

- (5 pts) (c) Give, with justification and base case(s), a recurrence relation that is suitable for a dynamic programming solution to this problem.

Solution: The base case in this situation is the vertex 1 where if the relation is represented by $P(i)$ then if the vertex is 1 the money Yousif can make is $P(i) = p(1)$ where $p(1)$ is the actual value of the price. To consider the relation we go back one vertex from i and know that the sum of the vertices payments of the previous vertices plus the current i is equal to the total payment or we can write this as for some $j \in V$ then $P(i) = \sum_{j \in V} P(j) + p(i)$ where $p(i)$ is the last index price.

- (7 pts) (d) Give a (bottom-up) dynamic programming algorithm, including pseudocode, that solves the “value version” of this problem in $O(n + m)$ time. (The “value version” is to find the maximum money that Yousif can earn, not necessarily a route that obtains it.) You may assume that for each $j \in V$, you are given a list of all its incoming edges $(i, j) \in E$.

Solution:

Pseudocode:

valueVersion(graph(V,E), prices)

//get the number of vertices in the graph that are possible

int number = get size of V

//create an array to hold each total from each path

array total (size number)

//have a nested for loop that goes through all the vertices in the outer loop and adds the total money from the previous vertices to the specific vertex. for(1:number)

int maxmoney

for(1:vertex i)

add each previous vertex sum to the current max sum, push total back to the array

outside of the for loop find the max money route from the array

- (2 pts) (e) Briefly describe in words (no pseudocode or correctness/runtime analysis needed) how to extend the above algorithm to output an optimum route.

Solution: To extend the algorithm to make the optimum route each time the vertices are gone through then an array of arrays can be made to store the vertices of the location of the optimal price. Then along with the max price the array corresponding to the route taken can be output.

6. Fixed-pattern paths.

Let $G = (V, E)$ be a graph with n vertices and m edges. Also, suppose that each edge is colored either black or white. We say that a path $P = (e_1, \dots, e_k)$ of edges has *color pattern* $(c_1, \dots, c_k)$, where each $c_i \in \{\text{black}, \text{white}\}$, if each e_i has color c_i . In this problem, you will design an algorithm that, given two vertices s, t and a desired color pattern $(c_1, \dots, c_k)$, determines in $O(mk)$ time whether there exists a path from s to t with that color pattern. Note that paths in this problem need not be *simple*; they can visit vertices or edges more than once.

- (10 pts) (a) Derive a recurrence relation, including base case(s), for this problem. Briefly justify its correctness.

Solution: The recurrence relation can be represented by the notion that there are two vertices and a color pattern that the two vertices should create with the path. We can represent the existence of this path as $P(s, t, c)$ where s and t are the vertices and c is the color pattern desired. The base cases in this case are if there is no input color pattern $P(s, t, 0)$ in which case if $s = t$ then is true as there is no color pattern for the same vertex. If for $P(s, t, 0)$ $s \neq t$ then this is false as there should be some pattern between two different vertices on an edge. The recurrence relation can be represented using dynamic programming relations as $P(s, t, c-1)$ where if there is an edge between s and t then the last color pattern crossed if this is not a simple path should be the same color pattern as c .

- (5 pts) (b) Give a bottom-up dynamic programming algorithm, including pseudocode, for this problem that has running time $O(k(n+m))$ (or just $O(km)$ under the assumption that $m \geq n$).

Solution:

Pseudocode:

```
colorPattern(graph(V,E), s, t, c[1...k])
get the number of colors and number of vertices in the graph
k = number of colors
n = number of vertices
create an array P to store 3 different variables [s][t][c]
set the base cases with a nested loop going through all combinations of vertices
for all s in n
  for all t in n
    P[s][t][0] = true if s = t otherwise false
//use bottom up
```

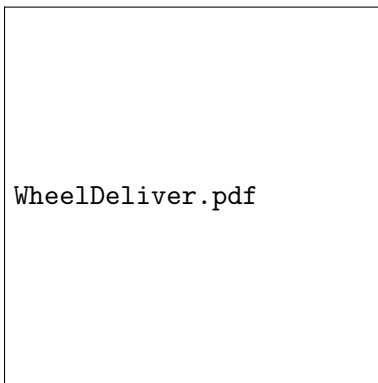


Figure 1: Two examples of routes for the two riders. The origin denotes the starting point p_0 . One rider's route is shown with solid lines and the other with dashed lines. The route on the left is fair because points are visited by increasing indices. The route on the right is not fair because, for example, point p_4 is visited before point p_1 .

```

for all colors from 1 - k
for all s in n
for all t in n
P[s][t][c] = P(s, t, c - 1) as long as the last color c - 1 is equal to the intended color of
this current k color index
if P[s][t][c] intended from the input exists in the array then return true otherwise return
false.

```

7. Optional extra credit: Wheel deliver.

Julie and Daphne have started a new unicycle-based meal delivery business called “Wheel Deliver” that works as follows. Residents of Ann Arbor submit their requests for a meal each day before noon using the Wheel Deliver App. In the afternoon, Julie and Daphne cook up a delicious meal in their kitchen and deliver the food on their unicycles.

Let n be the number of customers on a given day and $P = (p_1, \dots, p_n)$ be the list of locations of these customers as points in the 2D plane, sorted in the order in which the customers submitted their requests: p_1 is the location of the first person to place an order, and p_n is the location of the last person to place an order. Let p_0 denote the location of the kitchen from which Julie and Daphne depart. Let $d(p_i, p_j)$ denote the distance between p_i and p_j .

Julie and Daphne wish to split up the list of locations P , not necessarily evenly, so that each delivery is made by exactly one of them. In addition, “Wheel Deliver” has a fairness rule that goes like this: If customers at p_i and p_j for $i < j$ are assigned to the same unicycle rider, then the delivery to the customer at p_i must occur before the delivery to the customer at p_j —even if that makes the trip longer. (If p_i and p_j are assigned to different riders, then we don’t care which delivery occurs first.) Subject to these constraints, the objective is to minimize the total distance travelled by the two unicycle riders.

Julie and Daphne have hired you to design an efficient algorithm that takes as input an array

of points $P = (p_1, p_2, \dots, p_n)$, sorted by the times in which their orders arrived, and outputs the sequences of delivery points for the two riders.

(4 EC pts)

- (a) Derive a recurrence relation, including base case(s), that is suitable for an efficient dynamic-programming solution for this problem. Briefly justify its correctness.

Solution: A recurrence relation can be noted as $P(d, j)$ where this represents the minimum total distance traveled by the riders for a trip that has two riders where the distance each travel is modeled by d and j . For the base cases we can say that each base case represents the distance if only one rider takes both destinations. Therefore we can have either $P(d, 0)$ or $P(0, j)$ where Daphne is the rider in the first case and Julie is the rider in the second case. We can see that the recurrence takes into account the different cases, so the minimum of the two different cases can be the relation. Therefore we have the minimum distance as the $\min(P(d-1, j) + \text{distancebetween } p_{d-1}, p_d, P(d, j-1) + \text{distancebetween } p_{j-1}, p_j)$ where we have the previous distance between the points of delivery plus the last relation recurrence for each rider as a possibility and return the minimum distance of which rider traveled less.

(4 EC pts)

- (b) Give a (bottom-up) dynamic programming algorithm, including pseudocode, for this problem that has running time $O(n^2)$.

Solution: